KEEBOX

Authentication Architecture Specification

Registration, login, PIN security, OTP verification, recovery, JWT sessions, and local authentication

Specification version 1.0 | 03 September 2026

Core rule: account entry is protected by backend authentication, while routine app unlocking is local and remains fully usable without internet access.

This document consolidates the agreed Keebox authentication design. It is intended to act as the implementation contract for the Flutter client and Django Ninja backend.

1. Executive Summary

Keebox uses four distinct authentication mechanisms. Each mechanism has a separate responsibility and must not be conflated with the others.

Mechanism	Credential / Input	Verification Location	Primary Purpose
Account authentication	Email + password	Backend	Prove ownership of the Keebox account.
OTP authentication	OTP challenge ID + OTP code	Backend	Verify email access during registration and recovery flows.
Server PIN verification	PIN code	Backend	Complete login and manage the account-wide PIN.
Local PIN authentication	PIN code	Flutter device only	Unlock an already provisioned Keebox installation without internet.

The normal login process contains no OTP stage. It consists only of email/password authentication followed by server PIN verification. OTP is used for registration and recovery operations.

Password, PIN, and KBKey are independent security entities. Changing or resetting a password or PIN must never rotate or invalidate the user's KBKey.

2. Normative Authentication Decisions

- Registration starts with first name, last name, email, and password. The PIN is created only after registration OTP verification.

- Registration OTP verification uses a shared OTPVerification implementation also used by password recovery and PIN recovery.

- Successful registration is completed only after /create-pin succeeds.
- Normal login is email + password followed by PIN verification. Login does not use OTP.
- A Login Challenge ID tracks the multi-stage login attempt between password verification and PIN verification.
- JWT access and refresh tokens are issued only after the complete registration or login flow succeeds.
- The final successful registration/login response includes first name, last name, email, KBKey, access token, and refresh token.
- The raw PIN is never stored in the backend database or device secure storage.
- The backend stores a slow password-hash representation of the PIN, additionally protected with a server-side PIN pepper.
- Flutter stores a local PIN verifier and related parameters in secure storage, not the raw PIN.
- Routine app unlocking uses the local verifier only and does not contact the backend.
- Forgot-password and forgot-PIN are separate recovery flows and both use the shared OTP subsystem.

3. Terminology

Term	Definition
Registration Challenge	Tracks one incomplete account-registration process from initial submission through OTP verification and PIN creation.
Login Challenge	Tracks one login process after password verification and before PIN verification completes the session.

Term	Definition
OTP Challenge	One-time email-code challenge used by registration and recovery flows.
Recovery Challenge	Tracks a password-reset or PIN-reset process independently from the OTP challenge used to authorize it.
KBKey	The user's stable Keebox Key. Returned only after full registration/login and used by the client to encrypt/decrypt KBoxes.
KMKey	The Keebox Master Key held by the server and used to protect stored KBKeys.
Local PIN verifier	A one-way Argon2id-derived representation of the PIN used to unlock the app locally.

4. Challenge Context Architecture

Keebox does not store a shared authentication-purpose value. The concrete challenge model and its explicit parent relationship identify the workflow context without duplicating information.

4.1 Context ownership rules

- RegistrationChallenge represents registration and therefore has no purpose field.
- LoginChallenge represents normal login and therefore has no purpose field.
- RecoveryChallenge will use a recovery-specific type only when password and PIN recovery must be distinguished.
- OTPVerification belongs to exactly one registration or recovery context; that relationship identifies why the OTP exists.
- Normal login never creates an OTPVerification record.

4.2 Choice organization

All reusable choice classes live in apps.core.choices. Only choices required by the model currently being implemented are added. The first is RegistrationStatus.

```
RegistrationChallenge -> RegistrationStatus
OTPVerification -> parent registration or recovery relationship
No shared authentication-purpose enum or database field
```

5. Registration Status Choice

RegistrationStatus defines the complete lifecycle of a pending registration. It is implemented in apps.core.choices using Django's TextChoices interface.

```
class RegistrationStatus(md.TextChoices):
OTP_PENDING = ("otp_pending", "OTP pending")
OTP_VERIFIED = ("otp_verified", "OTP verified")
COMPLETED = ("completed", "Completed")
EXPIRED = ("expired", "Expired")
CANCELLED = ("cancelled", "Cancelled")
```

5.1 Registration state transitions

- OTP_PENDING transitions to OTP_VERIFIED after successful email OTP verification.
- OTP_VERIFIED transitions to COMPLETED only when PIN creation and complete User creation succeed.
- OTP_PENDING and OTP_VERIFIED may transition to EXPIRED or CANCELLED.
- COMPLETED, EXPIRED, and CANCELLED are terminal states and cannot be replayed.

A registration challenge expires exactly 30 minutes after it is created. The server clock is authoritative.

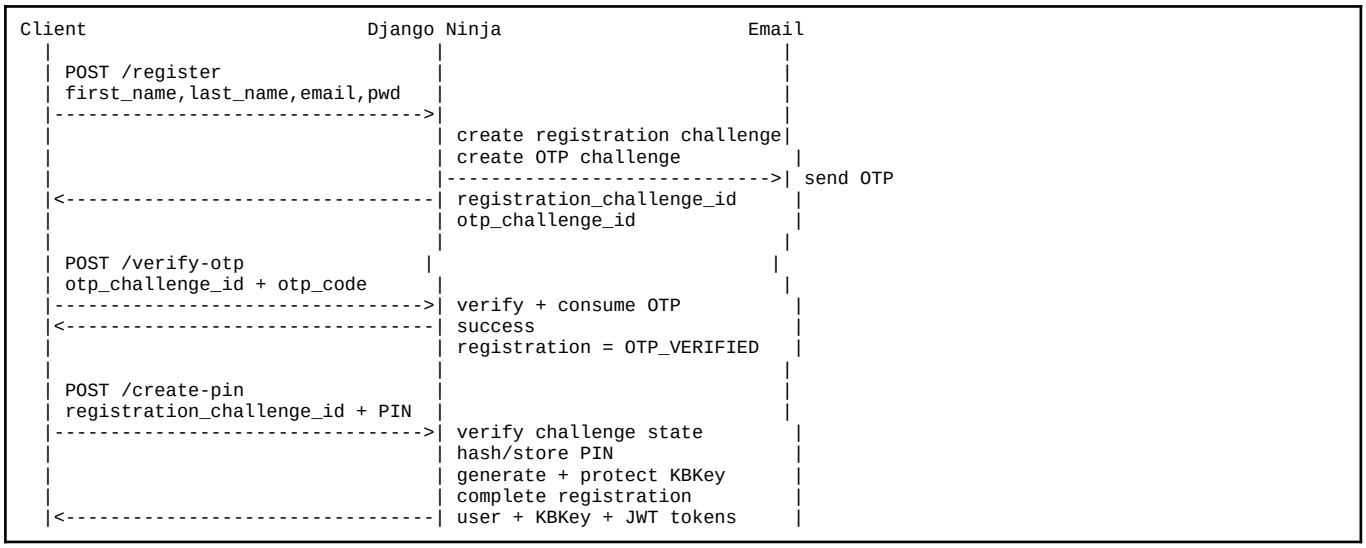
6. Registration Architecture

6.1 Input

Field	Requirement
first_name	Required.
last_name	Required.
email	Required; normalized and unique.
password	Required; validated by the server password policy.

The PIN is intentionally not part of /register. It is created only after the email OTP has been verified.

6.2 Sequence



6.3 /register response

```
{
  "registration_challenge_id": "<opaque UUID>",
  "otp_challenge_id": "<opaque UUID>",
  "email": "user@example.com",
  "otp_expires_at": "..."
}
```

The response does not contain JWT tokens or KBKey because the account is not fully registered yet.

6.4 Shared /verify-otp behavior during registration

The client submits only the OTP challenge ID and code. The OTP record already knows its purpose and owning registration context. A successful verification marks the OTP consumed and transitions the related RegistrationChallenge to OTP_VERIFIED.

```
POST /verify-otp
{
  "otp_challenge_id": "...",
  "otp_code": "..."
}
```

6.5 /create-pin and registration completion

/create-pin must require the Registration Challenge ID. It must not accept an email address as sufficient proof that registration OTP verification happened.

- Confirm the registration challenge exists and has not expired.
- Require status OTP_VERIFIED.
- Validate the PIN against the centralized PIN policy.
- Hash the PIN using the server PIN hashing strategy.
- Generate the user's one-time stable KBKey.
- Encrypt/protect the KBKey using KMKey before database storage.
- Mark registration complete.
- Issue access and refresh JWTs.
- Return the final authenticated response.

```
{
  "first_name": "Nelson",
  "last_name": "...",
  "email": "nelson@example.com",
  "kbkey": "KBK-....................-....................",
  "access_token": "...",
  "refresh_token": "..."
}
```

7. Login Architecture

Normal Keebox login does not use OTP. It consists of email/password verification followed by PIN verification.

7.1 Login initiation

```
POST /login
{
  "email": "nelson@example.com",
  "password": "..."
}
```

If the credentials are valid, the backend creates a LoginChallenge. The challenge represents the fact that the account password has been verified but the PIN requirement is still outstanding.

```
{
  "login_challenge_id": "<opaque UUID>",
  "expires_at": "..."
}
```

7.2 Why the Login Challenge ID exists

- It binds PIN verification to a specific password-authenticated login attempt.
- It prevents /verify-pin from accepting an email + PIN as a standalone login mechanism.
- It gives the backend an expiration window for completing the second stage.
- It provides a location for failed PIN attempt counters and temporary login lock state.
- It avoids issuing normal JWT credentials before the entire login process is complete.

7.3 PIN completion

```
POST /verify-pin
{
  "login_challenge_id": "...",
  "pin_code": "..."
}
```

The backend verifies the challenge state and submitted PIN. On success it marks the login challenge complete and returns the same final authenticated payload used after successful registration.

```
Email + password
      |
      v
LoginChallenge(PASSWORD_VERIFIED)
      |
      v
PIN verification
      |
      v
LoginChallenge(COMPLETED)
      |
      v
first_name + last_name + email + KBKey + access_token + refresh_token
```

8. PIN Management Architecture

The server endpoints /create-pin, /verify-pin, and /update-pin are PIN management operations. They do not replace local PIN authentication and must never become a network dependency for routine app unlocking.

Endpoint	Primary Context	Authorization Context
/create-pin	Initial registration only.	RegistrationChallenge in OTP_VERIFIED state.
/verify-pin	Complete normal login.	LoginChallenge in PASSWORD_VERIFIED state.
/update-pin	User intentionally changes a known PIN.	Authenticated JWT + current PIN.

8.1 PIN update

```
POST /update-pin
Authorization: Bearer <access token>

{
  "current_pin": "...",
  "new_pin": "..."
}
```

A successful PIN change updates the server verifier and the current device local verifier. KBKey is unchanged and no KBox re-encryption is required.

PIN changes should require network access. Changing only the local verifier while offline would desynchronize the local PIN from the server PIN used during the next login.

9. Server-Side PIN Protection

The server never stores the raw PIN. Because a PIN has a much smaller search space than a normal password, the stored verifier must use a deliberately expensive password-hashing algorithm and a server-side secret pepper.

```
PIN
 +
KEEBOX_PIN_PEPPER
  |
  v
peppered PIN material
  |
  v
Argon2id + random salt
  |
  v
stored pin_hash
```

KEEBOX_PIN_PEPPER is a separate server secret. It must not be the KMKey, Django SECRET_KEY, JWT signing secret, or any other unrelated key.

9.1 Reference backend helper

```
import hashlib
import hmac
from django.conf import settings
from django.contrib.auth.hashers import make_password, check_password



def _pepper_pin(pin: str) -> str:
    return hmac.new(
        key=settings.KEEBOX_PIN_PEPPER.encode("utf-8"),
        msg=pin.encode("utf-8"),
        digestmod=hashlib.sha256,
    ).hexdigest()



def hash_pin(pin: str) -> str:
    return make_password(_pepper_pin(pin), hasher="argon2")



def verify_pin(pin: str, encoded_hash: str) -> bool:
    return check_password(_pepper_pin(pin), encoded_hash)
```

Django should be configured with Argon2PasswordHasher support. Account passwords should continue to use Django's normal set_password/check_password API rather than custom hashing logic.

9.2 PIN rate limits and lockout

- Limit failed /verify-pin attempts per LoginChallenge.
- Rate-limit requests by account and network source where practical.
- After the configured threshold, temporarily lock or terminate the login challenge.
- Never expose whether a failure came from a malformed challenge, wrong PIN, or account state in a way that assists attackers.
- Local PIN verification should also add increasing delay or local lockout after repeated failures.

10. Local PIN Authentication

After a successful registration or login, Flutter provisions local authentication. From that point onward, opening Keebox does not require the server simply to verify the PIN.

```
Normal app opening
      |
      v
```

```
Show local lock screen
      |
      v
User enters PIN
      |
      v
Derive candidate local verifier
      |
      v
Compare locally
      |
      +-- correct --> unlock Keebox
      |
      +-- wrong ----> deny + throttle

No HTTP request is required.
```

10.1 Local secure-storage contents

Item	Storage Recommendation	Notes
Local PIN verifier	flutter_secure_storage	Store only the derived verifier/hash, never the raw PIN.
PIN salt	flutter_secure_storage	Random per-device salt used for local Argon2id derivation.
PIN parameters	flutter_secure_storage or app constants	Memory, iterations, parallelism, version.
KBKey	flutter_secure_storage	Protect locally after successful registration/login.
Refresh token	flutter_secure_storage	Long-lived session secret.
Access token	Memory preferred; secure storage if persistence is required	Short-lived JWT.

10.2 Local verifier reference implementation

```
import 'dart:convert';
import 'dart:math';
import 'package:cryptography/cryptography.dart';
import 'package:flutter_secure_storage/flutter_secure_storage.dart';


final _storage = FlutterSecureStorage();
final _argon2 = Argon2id(
  memory: 19 * 1024,
  parallelism: 1,
  iterations: 2,
  hashLength: 32,
);

Future<List<int>> deriveLocalPinVerifier(
  String pin,
  List<int> salt,
) async {
  final key = await _argon2.deriveKeyFromPassword(
    password: pin,
    nonce: salt,
  );
  return key.extractBytes();
}
```

The parameter values above are implementation examples rather than immutable protocol constants. Benchmark them on the Android devices Keebox intends to support and version the local verifier parameters so they can be upgraded later.

11. Shared OTP Subsystem

Keebox uses one OTPVerification model for registration and recovery. Normal login does not use OTP. The owning challenge relationship identifies the workflow context.

```
Registration -> RegistrationChallenge -> OTPVerification
Normal login -> LoginChallenge -> no OTP verification
Password reset -> RecoveryChallenge -> OTPVerification
PIN reset -> RecoveryChallenge -> OTPVerification
```

11.1 OTPVerification logical fields

```
id: UUID
user_id: UUID | None
email: str
registration_challenge_id: UUID | None
recovery_challenge_id: UUID | None
code_hash: str
status: OTP status choice
attempt_count: int
resend_count: int
last_sent_at: datetime
expires_at: datetime
consumed_at: datetime | None
Rule: exactly one parent challenge relationship must be present.
```

11.2 /verify-otp

```
POST /verify-otp {"otp_challenge_id": "...", "otp_code": "..."}
```

- Reject expired, locked, or consumed verification records.
- Compare the submitted code with the stored one-way verifier.
- Consume the code on success and advance its owning challenge atomically.

11.3 /resend-otp

The client submits the existing OTP Challenge ID. If the challenge is eligible for resend, the server generates a new OTP, replaces the old OTP verifier, resets the code expiry, increments resend_count, sends the new email, and keeps the same challenge ID. The previously sent OTP immediately becomes invalid.

```
POST /resend-otp
{
  "otp_challenge_id": "..."
}

Response:
{
  "otp_challenge_id": "...",
  "otp_expires_at": "...",
  "resend_available_at": "..."
}
```

12. Forgotten Account Password

Password recovery proves control of the registered email address and then allows the user to choose a new account password. KBKey and PIN are not modified.

```
POST /forgot-password
email
  |
  v
Create RecoveryChallenge(PASSWORD_RESET)
  |

Create OTPVerification(PASSWORD_RESET)

  v
Send OTP
  |
  v
Return recovery_challenge_id + otp_challenge_id
  |
  v
POST /verify-otp
  |
  v
RecoveryChallenge = OTP_VERIFIED
  |
  v
POST /reset-password
recovery_challenge_id + new_password
  |
  v
Replace password hash
  |
  v
RecoveryChallenge = COMPLETED
  |
  v
Return to normal login
```

12.1 /forgot-password

```
POST /forgot-password
{
  "email": "nelson@example.com"
}
```

For anti-enumeration, the public response should remain generic when practical. Internally, an OTP is only sent when the account is eligible for recovery.

12.2 /reset-password

```
POST /reset-password
{
  "recovery_challenge_id": "...",
  "new_password": "..."
}
```

The recovery challenge must be PASSWORD_RESET, OTP_VERIFIED, unexpired, and unused. The new password is stored using Django set_password(). The endpoint returns success but does not issue a normal logged-in JWT session. The user proceeds through /login and /verify-pin.

Password reset does not rotate KBKey. Previously encrypted KCredentialBoxes and KNoteBoxes remain decryptable with the same KBKey.

13. Forgotten Lock-Screen PIN

A forgotten PIN cannot be recovered because the raw PIN is never stored. Keebox replaces it only after strong account verification.

The agreed recovery proof is account password plus email OTP.

```
POST /forgot-pin
email + password
  |
  v
Verify account password
  |
  v
Create RecoveryChallenge(PIN_RESET)
  |

Create OTPVerification(PIN_RESET)

  v
Send OTP
  |
  v
POST /verify-otp
  |
  v
RecoveryChallenge = OTP_VERIFIED
  |
  v
POST /reset-pin
recovery_challenge_id + new_pin
  |
  v
Hash/store new PIN
  |
  v
Increment pin_version
  |
  v
Update local PIN verifier on current device
```

13.1 /forgot-pin

```
POST /forgot-pin
{
  "email": "nelson@example.com",
  "password": "..."
}
```

13.2 /reset-pin

```
POST /reset-pin
{
  "recovery_challenge_id": "...",
  "new_pin": "..."
}
```

If recovery was started from the local lock screen on an already provisioned installation, Flutter can create the new local PIN verifier immediately after the server accepts the new PIN. On a fresh installation or logged-out device, the user completes normal login after the reset.

PIN reset does not rotate KBKey and does not require KBox re-encryption.

14. If Both Password and PIN Are Forgotten

1. Complete forgot-password using email OTP and choose a new password.

2. Start forgot-PIN using the newly established password.

3. Verify the PIN-reset OTP and choose a new PIN.

4. Complete normal login with email + new password, then verify the new PIN.

The user's KBKey remains unchanged throughout the entire sequence, so backed-up KBoxes remain recoverable.

15. JWT Session Architecture

Keebox uses django-ninja-jwt for normal authenticated API sessions. Access and refresh tokens are issued only after the full authentication flow is complete.

Point in Flow	Issue Access/Refresh JWTs?
After /register	No
After registration /verify-otp	No
After registration /create-pin	Yes
After /login email/password validation	No
After login /verify-pin	Yes
After /reset-password	No; return to normal login
After /reset-pin	No new normal session unless the endpoint is already called in an authenticated PIN-change context

15.1 Final authentication response

```
{
  "user": {
    "first_name": "Nelson",
    "last_name": "...",
    "email": "nelson@example.com"
  },
  "kbkey": "KBK-....................-....................",
  "access_token": "...",
  "refresh_token": "..."
}
```

The token refresh endpoint should refresh authentication tokens only. It should not re-send KBKey on every refresh. If the client no longer has its locally protected KBKey, a fresh login is the clean re-provisioning path.

16. Proposed Logical Data Models

These logical models preserve atomic account creation: no User exists until registration OTP verification and PIN creation have both succeeded.

16.1 User authentication fields

```
id: UUID
first_name: str; last_name: str; email: normalized unique str
password: Django-managed password hash
pin_hash: Argon2-based server PIN verifier
pin_version: int
encrypted_kbkey: bytes; kbkey_nonce: bytes
kbkey_encryption_version: int
```

16.2 Challenge models

```
class RegistrationChallenge:
id: UUID
first_name: str; last_name: str; email: str
password_hash: str
status: RegistrationStatus
expires_at: datetime # created_at + 30 minutes
completed_at: datetime | None
class LoginChallenge:
id: UUID; user_id: UUID; status: LoginStatus
failed_pin_attempts: int; expires_at: datetime
completed_at: datetime | None
class RecoveryChallenge:
id: UUID; user_id: UUID; recovery_type: RecoveryType
status: RecoveryStatus; expires_at: datetime
completed_at: datetime | None
class OTPVerification:
id: UUID; user_id: UUID | None; email: str
registration_challenge_id: UUID | None
recovery_challenge_id: UUID | None
code_hash: str; status: OTPStatus
attempt_count: int; resend_count: int
last_sent_at: datetime; expires_at: datetime
consumed_at: datetime | None
```

17. Endpoint Contract Summary

Method	Endpoint	Request	Successful Result
POST	/register	first_name, last_name, email, password	registration_challenge_id + otp_challenge_id
POST	/verify-otp	otp_challenge_id, otp_code	Consumes OTP and advances parent challenge
POST	/resend-otp	otp_challenge_id	New OTP sent; same OTP challenge ID
POST	/create-pin	registration_challenge_id, pin_code	Completes registration; user + KBKey + JWTs
POST	/login	email, password	login_challenge_id
POST	/verify-pin	login_challenge_id, pin_code	Completes login; user + KBKey + JWTs
POST	/update-pin	current_pin, new_pin + JWT	Updates server PIN + pin_version
POST	/forgot-password	email	recovery_challenge_id + otp_challenge_id
POST	/reset-password	recovery_challenge_id, new_password	Password replaced; login required
POST	/forgot-pin	email, password	recovery_challenge_id + otp_challenge_id
POST	/reset-pin	recovery_challenge_id, new_pin	PIN replaced + pin_version incremented
POST	/token/refresh	refresh token	New JWT token(s); no KBKey

18. PIN Versioning and Multi-Device Behavior

An account-wide PIN can be cached as a local verifier on more than one device. This creates an unavoidable offline-revocation limitation: a device that is fully offline cannot instantly learn that the PIN changed elsewhere.

A lightweight pin_version field makes this behavior manageable.

```
Server User.pin_version = 4

Device A local metadata:
  pin_version = 4   -> verifier current

Device B local metadata:
  pin_version = 3   -> stale once device checks server
```

- Increment pin_version after /update-pin and /reset-pin.
- Store the currently provisioned pin_version beside the local verifier.
- When the app is online and already performing authenticated network activity, compare the server pin_version with the local version.
- If the versions differ, require re-provisioning of local PIN authentication before treating the old verifier as current.
- A completely offline device may still accept its previously provisioned local PIN until it reconnects. Instant remote revocation is impossible without sacrificing offline unlock.

19. Challenge Security Rules

- Use UUIDs or equally unguessable challenge identifiers.
- Every RegistrationChallenge, LoginChallenge, RecoveryChallenge, and OTPVerification expires.
- OTP verification and recovery completion cannot be replayed.
- Every endpoint validates the exact prerequisite state before an atomic transition.
- OTP and PIN verification enforce attempt limits; OTP resend enforces cooldown and invalidates the old code.
- Identity and ownership come from server-side challenge relationships, never arbitrary completion-request input.

20. Recommended Challenge Lifetimes

Challenge timing is centralized in settings and evaluated using the server clock. Registration has a fixed 30-minute timeout.

```
AUTH_REGISTRATION_CHALLENGE_TTL = timedelta(minutes=30)
AUTH_LOGIN_CHALLENGE_TTL = ...
AUTH_RECOVERY_CHALLENGE_TTL = ...
AUTH_OTP_TTL = ...
AUTH_OTP_RESEND_COOLDOWN = ...
AUTH_OTP_MAX_ATTEMPTS = ...
```

```
AUTH_PIN_MAX_ATTEMPTS = ...
```

Challenge expiration should be evaluated by the server clock. The mobile client may display countdowns for UX, but it does not decide whether a challenge is valid.

21. Security Boundaries and Secret Separation

Secret / Value	Where It Exists	Must Not Be Reused As
Account password	User input; Django password hash in database	PIN, KBKey, KMKey
PIN	Transient user input; hashed server verifier + local verifier	Password, KBKey
KBKey	Encrypted on server; securely provisioned to client	Password hash or PIN verifier
KMKey	Server secret store only	PIN pepper or JWT signing secret
KEEBOX_PIN_PEPPER	Server secret store only	KMKey or Django SECRET_KEY
JWT signing secret/key	Server secret store	KMKey or PIN pepper
OTP code	Email + transient input; only hash/verifier persisted	Persistent authentication secret

No security secret should be logged. This includes raw passwords, PINs, OTP codes, KBKeys, KMKey, PIN pepper, access tokens, and refresh tokens.

22. KBKey Provisioning Integration

Authentication is the controlled point at which a device receives the user's KBKey. The backend database stores an encrypted KBKey; KMKey is used server-side to recover the raw KBKey only after full registration or login succeeds.

```
Successful registration (/create-pin)
                OR
Successful login (/verify-pin)
                 |
                 v
Load encrypted KBKey
                 |
                 v
Decrypt with KMKey
                 |
                 v
Return KBKey over HTTPS
                 |
                 v
Flutter stores KBKey in secure storage
                 |
                 v
KBKey encrypts/decrypts KCredentialBox + KNoteBox
```

Recovery endpoints do not inherently need to return KBKey. Password recovery returns the user to normal login. PIN recovery can update the current device local verifier when appropriate, but a fresh/logged-out installation still obtains KBKey through normal login.

23. Logout and Local Authentication State

Logout must terminate the authenticated local security context. At minimum the client should clear access/refresh tokens, the locally provisioned KBKey, and the local PIN verifier. The treatment of the encrypted local KBox database is a separate data-retention policy and should be consistent with the backup/restore specification.

After logout, returning to the account requires the full normal login process: email + password, Login Challenge ID, and server PIN verification.

24. Django Ninja Reference Flow Skeleton

The following code is intentionally a service-layer skeleton. Production implementation should add transactions, rate limits, error mapping, challenge cleanup, audit events, and MongoDB-specific persistence details.

```
@router.post("/login")
def login(request, payload: LoginIn):
    user = authenticate_user(payload.email, payload.password)
    challenge = login_challenges.create(user=user)
    return {
        "login_challenge_id": str(challenge.id),
        "expires_at": challenge.expires_at,
    }



@router.post("/verify-pin")
def verify_login_pin(request, payload: VerifyPinIn):
    challenge = login_challenges.require_password_verified(
        payload.login_challenge_id
    )
    user = challenge.user

    pin_service.assert_valid(user, payload.pin_code, challenge)
    login_challenges.complete(challenge)

    kbkey = kbkey_service.decrypt_for_user(user)
    tokens = jwt_service.issue_pair(user)

    return build_auth_response(user, kbkey, tokens)
```

```
@router.post("/verify-otp")
def verify_otp(request, payload: VerifyOtpIn):
otp = otp_service.get_active(payload.otp_challenge_id)
parent = otp_service.resolve_parent(otp)
otp_service.verify_and_consume(otp, payload.otp_code)
if isinstance(parent, RegistrationChallenge):
registration_service.mark_otp_verified(parent.id)
elif isinstance(parent, RecoveryChallenge):
recovery_service.mark_otp_verified(parent.id)
else:
raise InvalidOtpContext()
return {"success": True}
```

25. Flutter Authentication State Machine

```
UNAUTHENTICATED
   |
   +-- registration --> REGISTRATION_OTP --> CREATE_PIN --> AUTHENTICATED
   |
   +-- login --------> LOGIN_PIN -----------------------> AUTHENTICATED
   |
   +-- password reset -> RECOVERY_OTP -> NEW_PASSWORD -> UNAUTHENTICATED
   |
   +-- pin reset ------> RECOVERY_OTP -> NEW_PIN ------> depends on device state

AUTHENTICATED / app locked
   |
   +-- local PIN correct --> UNLOCKED
   |
   +-- forgot PIN --------> online PIN recovery flow
```

25.1 Device provisioning after final auth

1. Persist the KBKey securely.

2. Create a fresh local random salt for PIN verification.

3. Derive and store the local PIN verifier using the PIN already validated by the server.

4. Persist the server pin_version with the local verifier.

5. Persist the refresh token securely and transition the app to the authenticated home state.

26. Error and Response Semantics

Condition	Recommended Client-Facing Category
Wrong email/password	INVALID_CREDENTIALS
Expired LoginChallenge	CHALLENGE_EXPIRED
Wrong server PIN	INVALID_PIN
PIN attempts exhausted	PIN_CHALLENGE_LOCKED
Wrong OTP	INVALID_OTP
Expired OTP	OTP_EXPIRED
OTP resend cooldown active	OTP_RESEND_COOLDOWN
Wrong challenge state	INVALID_CHALLENGE_STATE
Recovery already completed	CHALLENGE_CONSUMED
Network unavailable during local unlock	No error: local unlock does not use network

Error payloads should use stable machine-readable codes and safe user-facing messages. Internal exception details must not leak to clients.

27. Required Test Matrix

Registration

- Successful register → OTP → create PIN → JWT + KBKey.
- Wrong/expired OTP.
- create-pin before OTP verification is rejected.
- create-pin replay after completion is rejected.

- duplicate email registration behavior.

Login

- Correct password creates LoginChallenge but no JWT.
- Wrong password does not create usable challenge.
- Correct PIN completes challenge and returns JWT + KBKey.
- Expired LoginChallenge is rejected.
- PIN brute-force threshold locks/terminates the challenge.
- No OTP is generated or required during normal login.

Local unlock

- Valid local PIN unlocks while device has no internet.
- Wrong PIN is rejected locally.
- Raw PIN is absent from secure storage.
- Local verifier survives app restart but not explicit logout clearing.

Recovery

- Forgot password → OTP → reset → normal login.
- Forgot PIN requires valid account password before OTP issuance.
- Forgot PIN → OTP → reset updates server verifier and local verifier.
- Forgot both works sequentially.
- Neither password reset nor PIN reset changes KBKey.

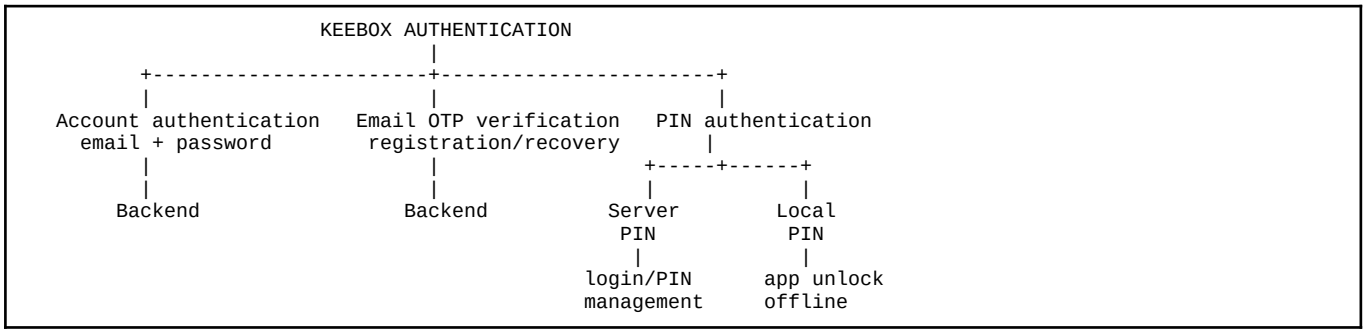
OTP

- REGISTRATION/PASSWORD_RESET/PIN_RESET purposes work.

- Normal login does not create or use an OTPVerification record.

- Resend invalidates the old code.
- Consumed OTP cannot be replayed.
- Attempt and resend limits are enforced.

28. Final Authentication Architecture



28.1 Registration

```
/register
  -> RegistrationChallenge + OTPVerification(REGISTRATION)
  -> /verify-otp
  -> /create-pin
  -> generate/protect KBKey
  -> access token + refresh token + user + KBKey
```

28.2 Login

```
/login (email + password)
   -> LoginChallenge
   -> /verify-pin
   -> access token + refresh token + user + KBKey

NO OTP in normal login.
```

28.3 Password recovery

```
/forgot-password
  -> RecoveryChallenge(PASSWORD_RESET)
  -> OTPVerification(PASSWORD_RESET)
  -> /verify-otp
  -> /reset-password
  -> normal login
```

28.4 PIN recovery

```
/forgot-pin (email + password)
  -> RecoveryChallenge(PIN_RESET)
  -> OTPVerification(PIN_RESET)
  -> /verify-otp
  -> /reset-pin
  -> update local verifier or proceed to normal login
```

28.5 Routine use

```
App opens
   -> local lock screen
   -> local PIN verifier
   -> home screen

Internet is not required.
```

The architecture deliberately makes account entry and account recovery server-controlled, while keeping everyday access to already provisioned encrypted KBoxes fast, local, and offline-capable.

29. Recommended Libraries and Platform Components

Area	Recommendation	Role
Backend authentication	Django authentication system	Password hashing, user identity, password validation.
API	Django Ninja	Authentication and recovery endpoints.
JWT	django-ninja-jwt	Access and refresh token sessions.
Password/PIN hashing	Django Argon2PasswordHasher + argon2-cffi	Slow one-way server verification.
OTP delivery	Brevo transactional email integration	Deliver registration and recovery codes.
Flutter secure persistence	flutter_secure_storage	Store local PIN verifier, KBKey, and refresh token.
Flutter local PIN KDF	cryptography package / Argon2id	Derive the local one-way PIN verifier.

30. Implementation Checklist

- Define RegistrationStatus in apps.core.choices using Django TextChoices.
- Implement RegistrationChallenge with atomic states and a fixed 30-minute expiration.
- Adapt the shared OTPVerification model and require exactly one owning registration or recovery relationship.
- Implement LoginChallenge and RecoveryChallenge with their model-specific choices.
- Implement /register, /verify-otp, /resend-otp, /create-pin, /login, /verify-pin, and /update-pin.
- Implement /forgot-password, /reset-password, /forgot-pin, and /reset-pin.
- Issue JWTs only after /create-pin or successful login /verify-pin.
- Return KBKey only at completed registration/login provisioning points.
- Keep the PIN pepper separate from KMKey, Django SECRET_KEY, and JWT secrets.
- Make routine local unlock network-independent and synchronize pin_version when online.
- Add rate limits, expiry, replay protection, resend cooldown, and secret-free audit logging.
- Test offline unlock, recovery, replay, brute-force limits, and KBKey invariance.